2) Regenerate Lab Activities

(i)-Activity 01:

```
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print(node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

print("Following is the Depth-First Search")
dfs(visited, graph, '5')
```

```
Following is the Depth-First Search
5
3
2
4
8
7
```

(ii)-Activity 02:

```
from collections import defaultdict

class Graph:

    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v, end=" ")
```

```
        for i in self.graph[v]:
            if not visited[i]:
                self.DFSUtil(i, visited)


    def DFS(self):
        V = max(self.graph) + 1

        visited = [False] * V

        for i in range(V):
            if not visited[i]:
                self.DFSUtil(i, visited)


g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 3)

print("Following is Depth First Traversal")
g.DFS()
```

```
Following is Depth First Traversal
0 1 2 3
```

3) Graded Lab Tasks

Lab Task 1:

```
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Fagaras': 99, 'Oradea': 151, 'Rimnicu Vilcea': 8
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni':
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
```

```
        'Eforie': {'Hirsova': 86},
        'Vaslui': {'Urziceni': 142, 'Iasi': 92},
        'Iasi': {'Vaslui': 92, 'Neamt': 87},
        'Neamt': {'Iasi': 87}
    }

    visited = set()
    path = []
    total_cost = 0
    found = False

    def dfs(node, goal):
        global found
        if found:
            return
        visited.add(node)
        path.append(node)

        if node == goal:
            found = True
            return

        for neighbor in sorted(graph[node]):  # alphabetical order
            if neighbor not in visited:
                dfs(neighbor, goal)
                if found:
                    return
        path.pop()

    dfs('Arad', 'Bucharest')

    cost = 0
    for i in range(len(path)-1):
        cost += graph[path[i]][path[i+1]]

    print("Path:", " -> ".join(path))
    print("Total cost:", cost)
```

```
Path: Arad -> Sibiu -> Fagaras -> Bucharest
Total cost: 450
```

Lab Task 02:

```
from collections import deque

graph = {
    "Arad": ["Sibiu", "Timisoara", "Zerind"],
    "Sibiu": ["Arad", "Fagaras", "Rimnicu Vilcea", "Oradea"],
    "Timisoara": ["Arad", "Lugoj"],
    "Zerind": ["Arad", "Oradea"],
    "Oradea": ["Zerind", "Sibiu"],
    "Fagaras": ["Sibiu", "Bucharest"],
    "Rimnicu Vilcea": ["Sibiu", "Pitesti", "Craiova"],
```

```
    "Lugoj": ["Timisoara", "Mehadia"],
    "Mehadia": ["Lugoj", "Drobeta"],
    "Drobeta": ["Mehadia", "Craiova"],
    "Craiova": ["Drobeta", "Rimnicu Vilcea", "Pitesti"],
    "Pitesti": ["Rimnicu Vilcea", "Craiova", "Bucharest"],
    "Bucharest": ["Fagaras", "Pitesti", "Giurgiu", "Urziceni"],
    "Giurgiu": ["Bucharest"],
    "Urziceni": ["Bucharest", "Hirsova", "Vaslui"],
    "Hirsova": ["Urziceni", "Eforie"],
    "Eforie": ["Hirsova"],
    "Vaslui": ["Urziceni", "Iasi"],
    "Iasi": ["Vaslui", "Neamt"],
    "Neamt": ["Iasi"]
}

def dfs(start, goal):
    stack = [(start, [start])]
    visited = set()
    nodes_visited = 0

    while stack:
        (vertex, path) = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            nodes_visited += 1
            if vertex == goal:
                return path, nodes_visited
            for neighbor in reversed(graph.get(vertex, [])):
                stack.append((neighbor, path + [neighbor]))
    return None, nodes_visited

def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = set()
    nodes_visited = 0

    while queue:
        (vertex, path) = queue.popleft()
        if vertex not in visited:
            visited.add(vertex)
            nodes_visited += 1
            if vertex == goal:
                return path, nodes_visited
            for neighbor in graph.get(vertex, []):
                queue.append((neighbor, path + [neighbor]))
    return None, nodes_visited

print("Case 1: BFS fewer nodes than DFS (Arad -> Sibiu)")
path_bfs, visited_bfs = bfs("Arad", "Sibiu")
path_dfs, visited_dfs = dfs("Arad", "Sibiu")
print("BFS Path:", path_bfs, "Nodes visited:", visited_bfs)
print("DFS Path:", path_dfs, "Nodes visited:", visited_dfs)

print("\nCase 2: BFS more nodes than DFS (Arad -> Bucharest)")
path_bfs, visited_bfs = bfs("Arad", "Bucharest")
```

```
path_dfs, visited_dfs = dfs("Arad", "Bucharest")
print("BFS Path:", path_bfs, "Nodes visited:", visited_bfs)
print("DFS Path:", path_dfs, "Nodes visited:", visited_dfs)
```

```
Case 1: BFS fewer nodes than DFS (Arad -> Sibiu)
BFS Path: ['Arad', 'Sibiu'] Nodes visited: 2
DFS Path: ['Arad', 'Sibiu'] Nodes visited: 2

Case 2: BFS more nodes than DFS (Arad -> Bucharest)
BFS Path: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest'] Nodes visited: 9
DFS Path: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest'] Nodes visited: 4
```